

Atlas Program Mathematics and Formal Logic

Source-derived technical extraction from Masterstep193_MULTILINGUAL_EPISTEMIC_STATE_REPAIRED_KEEPER.zip

Evidence boundary

This report describes mathematics, scoring rules, thresholds, boolean logic, set-like operations, state transitions, evidence sufficiency rules, matrix arbitration, recursive/governance invariants, and proof-oriented logic actually present in the supplied program package. It does not manufacture a single 'theory equation' where the implementation contains multiple independent mechanisms. The ZIP internally retains the Masterstep192 directory name; this report identifies the source package as Masterstep193 but preserves internal source paths exactly.

Important: this is a static code extraction, not a new proof run and not a closure declaration.

1. Route Confidence: Weighted Sum With Cap

The clearest explicit numeric scoring equation in the Cognition runtime is the route scorer.

$S_{\text{raw}} = 20 \cdot A + 10 \cdot G + 20 \cdot N + \sum(B_i)$ $S = \min(S_{\text{raw}}, 100)$ A = number of detected axes G = number of active signals $N = 1$ when return mode is non-neutral, otherwise 0 B_i = matched conditional bonus

Component	Weight / condition
Each detected axis	+20
Each active signal	+10
Non-neutral return mode	+20
worth + permanence	+10
identity + permanence	+10
hypervigilance signal	+5
shame_after_visibility signal	+5
Maximum	100
High confidence	score >= 70
Medium confidence	score >= 40
Low confidence	score >= 0

Confidence is explicitly non-authoritative: the registry states that route confidence is a structural signal, not a claim of truth.

Sources: Cognition/scoring/RouteScorer.js; CA/Continuity_Architecture/07 Atlas Response Construction/40 Obsidian Runtime Response Registry/route_score_registry.md

2. Closed-Loop Axis Pressure Mathematics

A separate backend closed-loop engine computes seven normalized pressure dimensions:

$AXES = \{SELF, OTHERS, RESPONSIBILITY, CONTROL, SAFETY, BELONGING, FUTURE_MOVEMENT\}$

Base accumulation is additive. For every evidence item in stream k :

```
score_axis <- score_axis + W[k, axis]
```

Evidence stream	Axis weights
stated_emotional_effects	SELF .25; SAFETY .20
uncertainty_language	CONTROL .30; SELF .15
assumption_markers	RESPONSIBILITY .35; OTHERS .15
action_impulses	CONTROL .20; FUTURE_MOVEMENT .15; OTHERS .10
unsupported_conclusions_quarantine	BELONGING .45; FUTURE_MOVEMENT .30
intensity_markers	SAFETY .08; FUTURE_MOVEMENT .08
relational_terms	OTHERS .12; BELONGING .10
temporal_pressure	FUTURE_MOVEMENT .16; CONTROL .08
lived_mechanism_signals	RESPONSIBILITY .38; CONTROL .22; FUTURE_MOVEMENT .18

Action-impulse subtype increments:

```
REPAIR: OTHERS += .18; RESPONSIBILITY += .12; CONTROL += .10 WITHDRAWAL: CONTROL += .20;
FUTURE_MOVEMENT += .18; SAFETY += .08 CLARIFICATION: CONTROL += .08; OTHERS += .06
```

Dynamic multipliers:

```
urgencyHits = count(today|tomorrow|now|right now|always|never) FUTURE_MOVEMENT *= 1 + min(.30,
.08*urgencyHits) if relationalHits > 1: OTHERS *= 1.15 BELONGING *= 1.12 if assumptionCount >
0 and unsupportedCount > 0: RESPONSIBILITY *= 1.10 BELONGING *= 1.10 FUTURE_MOVEMENT *= 1.08
final axis score = min(1, round(score, 2))
```

State classification:

```
ACTIVE_LOAD if score >= 0.65 SECONDARY_POSSIBLE if 0.10 < score < 0.65 CLEAN_BASELINE
otherwise
```

Source: runtime/backend/closedLoop/AxisEngineScoring.js

3. Semantic Attachment Candidate Scoring

Attachment-family candidates use a second explicit weighted count:

```
familyScore = 3*phraseHits + 2*emotionHits + 3*effectHits + 2*actionHits
```

Candidates with score <= 0 are discarded. Remaining candidates are sorted descending by score, with family ID as a deterministic tie-breaker. At most four active candidates are returned.

Context-before-route can override raw lexical advantage. In relational-response uncertainty, a broad ability candidate can be capped at 1, while tone_shift_relational_uncertainty is promoted to at least 8.

```
ability_score <- min(ability_score, 1) relational_uncertainty_score <- max(existing_score, 8)
```

Sources: runtime/semantics/SemanticAttachmentFamilyRegistry.js; runtime/semantics/ActiveAttachmentScorer.js

4. AXOL Detection as Ordered Set/Evidence Logic

The primary AxisEngine is not a probabilistic classifier. It is deterministic registry matching with priority ordering, deduplication, evidence-state tagging, and actor attribution.

```
rules := sort(registry.rules, ascending priority) for each matching rule: axes <-  
union_preserving_order(axes, rule.axes) patterns <- union_preserving_order(patterns,  
rule.patterns) evidence(axis) <- union(evidence phrases / regex hits) states(axis) <-  
unique(states of evidence hits) dominant_axis = axes[0] if any, else null
```

This means 'dominant' is priority/order-derived, not the maximum of a numeric axis probability in this engine.

Source: Cognition/engine/AxisEngine.js

5. Epistemic Evidence State

Evidence is tagged as asserted, negated, or uncertain. In the multilingual substrate the decision order is:

```
if clause contains an uncertainty marker -> uncertain else if clause contains a configured  
negator around evidence -> negated else -> asserted
```

The multilingual detector normalizes Unicode with NFKC, lowercases, collapses whitespace, applies token boundaries to very short Latin evidence (length <= 5), and otherwise permits substring/stem-family matching.

Source: Cognition/engine/MultilingualAxolSubstrate.js

6. Signal Detection Logic

Signal rules combine OR inside a phrase group and AND across required groups. Direct phrases are also required when present.

```
anyPhraseMatches(P) = OR(p in P: text contains p) allGroupsMatch(G) = AND(g in G:  
anyPhraseMatches(g.phrases)) ruleMatch = allGroupsMatch(rule.allGroups) AND  
anyPhraseMatches(rule.phrases) [when direct phrases exist]
```

Signals are deduplicated before return.

Source: Cognition/signals/SignalEngine.js

7. Context-Before-Route Evidence Sufficiency

The evidence-sufficiency matrix requires eight fields:

```
{surfaceEvent, attachedMeaning, unsupportedConclusion, emotionalPressure,  
restorationDirection, literalPracticalBoundary, competingMeanings, routeReadiness}
```

The route-readiness transition is deterministic:

```
if surfaceEvent missing OR attachedMeaning missing: readiness = HOLD else if any field missing  
OR unresolvedCompetition OR practicalHold: readiness = LOW_CONFIDENCE else: readiness =  
PROCEED sufficientForRoute = (readiness == PROCEED)
```

Unresolved competition occurs when competingMeanings has more than one entry and competitionResolved is false. A practical hold occurs when the literal boundary is literal/practical/instructional/task and reflectiveRouteJustified is false.

Source: runtime/understanding/ContextBeforeRouteEvidenceSufficiencyMatrix.js

8. Mixed-Context Separation Logic

The mixed-context matrix refuses to collapse multiple contexts into one route before their relationship is resolved.

```

mixed = contextCount > 1 unresolved = contexts where !evidenceSufficient OR
unresolvedCompetition OR practicalHold routeCollapseRisk = mixed AND (unresolvedCount > 0 OR
!contextRelationshipResolved) if mixed AND routeCollapseRisk: readiness =
HOLD_FOR_CONTEXT_SEPARATION else if mixed AND every context sufficient: readiness =
PROCEED_WITH_SEPARATED_ROUTE_CANDIDATES routeSelectionAllowed = mixed ? (readiness ==
PROCEED_WITH_SEPARATED_ROUTE_CANDIDATES AND contextRelationshipResolved) : (single context
AND context.evidenceSufficient)

```

Source: runtime/understanding/MixedContextEvidenceSeparationMatrix.js

9. Pressure / Performance / Identity Matrix

This matrix has an explicit evidence-count qualification and bounded confidence equation.

```

coreCount = countTrue( effort_or_completion_present, felt_insufficiency_after_effort,
performance_or_output_context_present) qualified = (coreCount == 3) AND not
evidence_owned_by_other AND not evidence_explicitly_rejected AND not
evidence_currently_revoked matched = qualified AND not relational_comparison_present AND not
relational_context_primary AND not literal_task_only_present

```

Confidence:

```

if matched: C = min(.94, .72 + .08*expectedSufficiency + .10*worthIdentityAttachment) else: C
= min(.69, .20*coreCount) C is rounded to 2 decimals

```

Thus a matched case starts at 0.72, reaches 0.80 with explicit expected sufficiency, 0.82 with explicit worth/identity attachment, and 0.90 with both. The .94 cap is present but unreachable from these two bonuses alone.

Source: runtime/understanding/PressurePerformanceIdentityMatrix.js

10. Response Quality as Conjunctive Gate

Response quality is a set of registry-defined predicates. A response passes only when no predicate produces an issue.

```
structureCount = max(paragraphSectionCount, lineSectionCount) minimum_structure_count:  
structureCount >= minimum minimum_character_length: response.length >= minimum regex_any:  
OR(pattern matches response) qualityPassed = (issues.length == 0)
```

Source: Cognition/quality/ResponseQualityChecker.js

11. Matrix Growth Governance as Seven-Gate Conjunction

A candidate matrix has seven required gates:

```
G = {distinctness, recurrence, boundary_safety, compatibility, proof_burden,  
rollback_readiness, human_approval} failed_gates = {g in G | evidence[g] != true}
```

Lifecycle transition:

```
if contradiction OR speculative: REJECTED else if overlap: MERGE_RECOMMENDED else if  
failed_gates nonempty OR one_off_prompt_only: QUARANTINED else: APPROVED_FOR_PROOF
```

Even APPROVED_FOR_PROOF cannot mutate the matrix immediately, cannot promote without a human, and cannot skip full proof. This is an authority invariant encoded as constant false/true capabilities, not merely documentation.

Source: runtime/governance/MatrixGrowthGovernanceContract.js

12. Crown Predicate Mathematics / Logic

Crown predicate results use a three-valued state set: {PASS, FAIL, UNKNOWN}. Inputs are deterministically serialized with sorted object keys and hashed with SHA-256.

```
digest(x) = SHA256(stableSerialize(x)) satisfies(result, requirement) = schema_match AND  
result == PASS AND predicate_id_match AND predicate_version_match AND (required_digest absent  
OR digest_match)
```

Source: runtime/governance/crown/CrownPredicateResult.js

13. Crown Proof Qualification and Mutation Adequacy

Crown's proof layer also encodes disagreement resolution and mutation-test adequacy. Mutation adequacy is exact counting:

```
survivors = count(witness where mutant_detected != true) killed = total - survivors adequacy =  
PASS iff survivors == 0
```

For implementation/oracle disagreement, matching results resolve directly; disagreement without PASS evidence freezes as CONFLICT; qualified decisive evidence may resolve to PASS or FAIL.

Source: runtime/governance/crown/CrownProofQualification.js

14. Program-Wide Formal Pattern

The implementation does not reduce to one scalar equation. Its recurring formal structure is a staged constrained decision system:

```
raw input -> deterministic evidence predicates -> ordered/deduplicated AXOL sets -> evidence  
state + attribution -> context sufficiency / separation gates -> weighted candidate or  
pressure scoring where applicable -> bounded confidence -> governance predicates -> response
```

quality predicates -> governed output

A compact abstraction faithful to the code is:

E = Extract(x) A = Axol(E) C = ContextSeparate(E, A) S = Score(C) [only in scoring subsystems]
R = Route(C, S) G = Governance(R, provenance, authority predicates) Q = Quality(G) Output
allowed only under the applicable gate conditions

This abstraction is explanatory notation for the implemented composition; it is not copied from a single source-code equation.

15. What Is Arithmetic vs. What Is Formal Logic

Mechanism	Type
RouteScorer	weighted arithmetic + cap + thresholds
AxisEngineScoring	weighted accumulation + multipliers + normalization + thresholds
Attachment scorer	weighted hit-count arithmetic
PressurePerformanceIdentity	boolean qualification + bounded confidence arithmetic
AxisEngine / AXOL	ordered-set logic + deterministic predicates
SignalEngine	AND/OR predicate logic
Context sufficiency	state-transition / boolean gate logic
Mixed-context matrix	set partition + boolean gate logic
Matrix growth	seven-gate conjunction + lifecycle state machine
Crown predicates	three-valued logic + SHA-256 provenance identity
Response quality	conjunction of registry predicates

16. Interpretation Boundary

The strongest source-supported description is that Atlas combines deterministic symbolic evidence extraction, weighted scoring in selected subsystems, bounded confidence, context-separation matrices, state machines, provenance hashing, and conjunctive governance gates. The program does not currently support a claim that all of Atlas is one novel arithmetic system, nor that its numeric weights were mathematically optimized. They are explicit engineered constants and rules in the implementation.

Likewise, a 100/100 proof result demonstrates passage of the defined proof suite for the tested package/environment. It is not a mathematical proof of universal correctness, completeness, or safety.

Primary source package: Masterstep193_MULTILINGUAL_EPISTEMIC_STATE_REPAIRED_KEEPER.zip